# PWA security when the server turns hostile

**A Progressive Web App cannot ultimately defend itself against a fully compromised hosting server.** This is the web platform's defining architectural limitation: the origin server delivers both code and data, and the browser grants unconditional trust to all code from the same origin. Every client-side defense — service worker caching, SRI, CSP, cryptographic verification — buys time or mitigates partial-compromise scenarios, but none survives an attacker who controls the server for more than roughly **24 hours**. The only web-platform mechanism specifically designed to break this dependency is Google's **Isolated Web Apps (IWA)** proposal, which replaces live server delivery with signed web bundles GitHub Chrome Developers — but it remains Chrome-only and experimental. For applications requiring genuine protection against server compromise, the industry consensus (Signal, Threema, and others) remains: ship native apps with OS-level code signing.

That said, a defense-in-depth strategy combining service worker caching, embedded hash verification, CSP enforcement, and client-side encryption can meaningfully raise the attack bar. Understanding exactly *where* each layer breaks is essential for building the strongest possible PWA.

---

## The service worker: a temporary shield, not a trust anchor

The service worker is the most powerful defensive primitive available to a PWA — and its most critical limitation is that **it cannot protect itself from replacement**. The W3C Service Worker specification's Update algorithm explicitly sets the request's service-workers mode to "none", which prevents the active service worker from intercepting the browser's fetch of its own replacement script. GitHub github This is an intentional security decision (preventing SWs from caching themselves forever), but it means a legitimate defensive SW cannot verify or block a malicious update served by a compromised server.

**Update triggers and timing** determine how quickly the attacker's code reaches the client. Every navigation to an in-scope page triggers an update check. Medium google Push events and background sync also trigger checks, throttled to once per 24 hours. Medium chrome The browser performs a **byte-for-byte comparison** of the fetched script against the installed version — a single byte difference triggers installation of the new version. Chrome Developers With the default updateViaCache: 'imports' setting (since Chrome 68), the top-level SW script request always bypasses the HTTP cache and hits the network directly. Chrome Developers chrome

The attacker's new service worker can call **self.skipWaiting()** in its install handler, immediately activating without waiting for existing tabs to close. bitsofcode Combined with clients.claim(), it takes control of all open pages instantly. MDN Web Docs Felix Gerschau The normal lifecycle safeguard — waiting for zero controlled clients — is completely bypassed. This means the practical **window of protection is measured in minutes to hours**, not the theoretical 24-hour maximum. The moment the user navigates, the compromised server delivers its payload.

During this window, however, the old SW functions perfectly. The Cache API has **no automatic expiration or freshness checks** — cached entries persist until code explicitly deletes them, the user clears browser data, or the browser evicts under storage pressure (web.dev) (Safari aggressively evicts data for origins unused for ~7 days; (MDN Web Docs) Chrome and Firefox are more generous). A well-designed cache-first SW continues serving verified cached assets for all interceptable requests. Service worker registrations and caches persist across browser restarts, providing continuity — but reopening the browser triggers a navigation, which triggers an update check.

A defensive SW can implement a **circuit breaker pattern**: maintain a hash manifest of all critical resources (baked into the SW at build time), verify every network response against expected SHA-256 hashes using the Web Crypto API, and fall back to cached versions on mismatch. Angular's [ngsw] and Blazor's PWA implementation both use this approach in production. (Swimburger) The circuit breaker state must be persisted to IndexedDB since SWs are event-driven and can be terminated at any time. But this defense evaporates the moment the SW itself is replaced.

---

## No code signing exists for service workers today

The web platform currently provides **no mechanism to cryptographically pin or verify a service worker script**. This is the critical missing primitive. Several proposals exist but none have been implemented:

**Issue #1680** on the W3C ServiceWorker repository (opened May 2023) proposes adding an [integrity] field to [navigator.serviceWorker.register()]. (github) This would allow the registering page to pin the SW script to a known hash, creating what the proposal calls a "programmable root of trust." (GitHub) The registering HTML would specify [navigator.serviceWorker.register('/sw.js', { integrity: 'sha384-...' })], and the browser would refuse to install any SW script that doesn't match. This remains an open proposal with no spec changes made.

**Issue #1720** proposes a [verify] event on [ServiceWorkerRegistration] that fires after download but before installation, allowing application code to accept or reject a new SW script — for instance by checking an Ed25519 signature delivered via HTTP header against a public key distributed via SRI-backed script. (GitHub) No spec work has occurred.

**Issue #1318** (opened 2018, closed) proposed allowing the active SW to intercept its own update fetch, (github) which would have enabled the SW to verify the replacement script's signature. This was rejected due to the risk of SWs that accidentally or maliciously cache themselves forever.

The [importScripts()] function also lacks an integrity parameter. A 2015 proposal (w3c/webappsec Issue #454) suggested extending [importScripts()] to accept integrity metadata, (GitHub) but it was never implemented. One mitigation: since Chrome 71, [importScripts()] is restricted to the SW's installation phase only, and imported scripts are cached and byte-compared on updates. (Chrome Developers +3)

---

## SRI and CSP provide layered but brittle protection

**Subresource Integrity works with service worker-served responses.** When an HTML page contains `<script integrity="sha384-..." src="app.js">` and the SW intercepts that fetch, the browser still validates the integrity of whatever response the SW provides via `respondWith()`. Firefox Bug 1393439 confirmed this behavior, Mozilla Bugzilla and Frederik Braun's SRI+SW challenge demonstrated it in practice. However, the SW cannot read the HTML's integrity metadata — `request.integrity` is always an empty string within the SW's FetchEvent context (w3c/ServiceWorker Issue #1507). The browser performs the check after receiving the SW's response, not inside the SW.

**SRI for ES modules** was a critical gap until 2024. Shopify drove the addition of an `"integrity"` field to import maps, which shipped in Chrome 127, Edge 127, and Safari 18. shopify Shopify Engineering This allows hash-based verification of dynamically imported modules — a significant improvement for modern JS-heavy PWAs.

The fatal weakness of SRI remains: **it cannot protect the HTML document itself**. The HTML specifies the integrity hashes. If the attacker serves new HTML, they simply omit the `integrity` attributes or point to malicious scripts with matching attacker-generated hashes. SRI defends against compromised CDNs and third-party scripts, Mozilla not a compromised first-party server.

**Content Security Policy via `<meta>` tags** in cached HTML pages is enforced by the browser regardless of how the page is delivered. A cached page with `<meta http-equiv="Content-Security-Policy" content="script-src 'sha256-ABC...'">` will only execute scripts whose content matches the specified hashes. A service worker can also inject CSP headers into synthetic Response objects, and browsers enforce these headers — Paul Kinlan (Google Chrome DevRel) demonstrated this pattern for nonce-based CSP in service workers. kinlan

The critical nuance: **`script-src 'self'` provides zero protection against a compromised server** because `'self'` refers to the origin, which the attacker now controls. Only **hash-based** `script-src` values (no `'self'`, no domain allowlists) lock down execution to specific known scripts. Multiple CSP policies stack by intersection — additional policies can only further restrict, never relax MDN Web Docs — so a meta-tag CSP cannot be weakened by an HTTP header CSP, though the attacker can serve entirely new HTML without the meta tag.

The `require-sri-for` CSP directive was experimentally implemented in Chrome 54 and Firefox but **has been removed from all browsers**. W3cubDocs Its successor, the `Integrity-Policy` HTTP response header, shipped in Chrome 127+ and Firefox 145. Mozilla Bugzilla However, since it's delivered as an HTTP header, a compromised server simply doesn't send it.

A "maximum protection" cached page would combine hash-only `script-src` in CSP (no `'self'`), SRI on all script/link tags, import maps with integrity for ES modules, and a service worker that injects CSP headers and verifies all fetched resources against baked-in hashes. **This entire defense collapses when the SW updates**, which happens within hours of server compromise.

---

## Client-side cryptography cannot verify its own code

The pattern of embedding a public key in the service worker at build time and verifying Ed25519 or ECDSA signatures on API responses is technically sound and **performant enough for production use**. Ed25519 is now supported in all major browsers (Safari 17.0, Firefox 129, Chrome 137) through the Web Crypto API, (MDN Web Docs) (Signalplus) with sub-millisecond verification, 32-byte keys, and 64-byte signatures. The W3C Web Crypto Level 2 spec (FPWD April 2025) formally includes Ed25519. (WICG)

This pattern protects against a specific and important threat model: **a compromised API backend with intact application code** (e.g., the app is cached in the SW, but the API server is returning tampered data). The SW verifies each API response's signature before processing it, falling back to cached data or showing an error on verification failure.

Against full server compromise, however, this defense fails because **the attacker can serve code without the verification logic** or replace the embedded public key with their own. The verification code cannot verify itself — this is the fundamental bootstrapping problem of web security.

**IndexedDB and all client-side storage are fully exposed** to compromised same-origin JavaScript. The same-origin policy, which normally provides strong isolation, becomes meaningless when the attacker *is* the origin. (Mozilla) `CryptoKey` objects marked `extractable: false` prevent key export but **do not prevent use** — compromised JS can use non-extractable keys as signing or decryption oracles for their designated operations. The raw key material lives on disk in the browser's IndexedDB storage files, potentially extractable through filesystem access.

Encrypting IndexedDB with password-derived keys (via PBKDF2 or Argon2) provides **defense at rest**: data cannot be decrypted without the user's password. But compromised JS can intercept the password at entry time, derive the key, and exfiltrate everything. The defense only holds if the user hasn't entered credentials during the compromised session.

---

## Isolated Web Apps: the only architecture designed for this threat

Google's **Isolated Web Apps (IWA)** proposal directly addresses the compromised-server threat model. IWAs replace live server delivery with **Signed Web Bundles** (`.swbn` files) containing all application resources, signed with Ed25519 or ECDSA P-256 keys held by the developer. (Chrome Developers) The app's identity is derived from the **public key hash**, not a domain name, and uses the `isolated-app://<base32-encoded-public-key>` URL scheme. (GitHub) (GitHub)

The security differences from normal PWAs are stark. IWAs enforce a strict CSP (`script-src 'self' 'wasm-unsafe-eval'; require-trusted-types-for 'script'`) with no `eval()`, no `new Function()`, no inline scripts. (Chrome Developers) Resources are never fetched over the network at runtime. (Chrome Developers) Updates are delivered as new signed bundles with monotonically increasing version numbers for downgrade protection. (GitHub) The IWA explainer states: *"This proposal addresses that vulnerability by defining a single moment at which the application is vulnerable to a network attacker, that is, the moment when a bundle is downloaded."* (GitHub)

**Implementation status**: IWAs are available in Chrome behind chrome://flags/#enable-isolated-web-apps, with active development targeting ChromeOS as the primary platform. Enterprise distribution is supported via managed Chrome policies. Integrity Block Version 2 has been required since Chrome M129. GitHub **No other browser vendor has committed to implementing IWAs** — Firefox and Safari show no support or signals. This single-vendor status severely limits IWA's viability as a general solution.

---

## Known attacks and the academic landscape

Academic research has extensively documented **service workers as attack vectors**, but defensive usage remains largely theoretical. Key research includes:

The **SW-XSS** paper (Chinprutthiwong et al., ACSAC 2020) found 40 real websites vulnerable to attacks where malicious code targets unsanitized parameters in importScripts(), enabling permanent client-side site takeover. ACM Digital Library The **"Sleeper Agents"** paper (Karami et al., NDSS 2021) demonstrated history-sniffing attacks through SW-based timing side channels, prompting Chromium to redesign site isolation. NDSS Symposium The **SoK: Workerounds** paper (Subramani et al., IEEE EuroS&P 2022) systematized all known SW attack vectors including botnet/DDoS, cryptomining, persistent XSS, and malicious extension-to-SW injection. Oaklandsok arXiv

The **Earth Wendigo** APT campaign (documented by Trend Micro, 2021) represents one of the first nation-state uses of service worker registration for credential harvesting and email exfiltration against a Taiwanese organization. Trend Micro

The **polyfill.io supply chain attack** (2024) directly demonstrated PWA caching risks: over 380,000 hosts embedded compromised scripts, Censys and remediation required clearing all caches including service worker caches. Wiz.io's analysis explicitly noted that "the risk only remains relevant so long as the script is cached (via CDN, service workers, browser cache, etc.)." Wiz

On the defensive side, the most significant project is **WEBCAT** (Web-based Code Assurance and Transparency), published by Giulio Berra of the Freedom of the Press Foundation (ePrint 2025/797). IACR WEBCAT uses Sigstore-signed manifests of all static assets, published to transparency logs. A Firefox extension verifies served code against signed manifests and **fails closed** — if verification fails, the page does not load. SecureDrop Proof-of-concept deployments exist for Element, Jitsi, Bitwarden, CryptPad, and GlobaLeaks. SecureDrop Meta's **Code Verify** extension (2022), built with Cloudflare, takes a similar but weaker approach — it shows traffic-light indicators rather than blocking execution. fb FB

Daniel Huigens (Proton's cryptography lead and W3C Web Crypto spec editor) presented a **Source Code Transparency** proposal at the W3C "Secure the Web Forward" workshop (September 2023), using transparency logs to ensure all users receive the same code — making targeted attacks detectable even if not preventable. W3C w3 This remains in early proposal stages.

---

### What a practical defense-in-depth strategy looks like

No single technique suffices. The strongest practical configuration combines multiple layers, each with understood failure modes:

- **Service worker with embedded hash manifest**: Cache all critical resources at install time with SHA-256 hashes baked into the SW script. Serve cache-first for all assets. Verify network responses against the manifest before caching. Fall back to cached versions on integrity failure. Call `navigator.storage.persist()` to reduce eviction risk. MDN Web Docs
- **Hash-only CSP injected by the SW**: The SW constructs Response objects with `Content-Security-Policy` headers using exclusively hash-based `script-src` values — never `'self'` or domain allowlists. This locks execution to known scripts even if some network bypass occurs.
- **SRI on all subresources with import map integrity**: Every `<script>` and `<link>` tag includes `integrity` attributes. Import maps specify hashes for all ES modules. This creates browser-enforced verification independent of the SW.
- **Ed25519 signature verification on API responses**: The SW embeds the developer's public key and verifies signed API payloads. This detects API-level tampering while the SW remains uncompromised.
- **Password-derived encryption for IndexedDB**: Derive AES-GCM keys from user credentials via PBKDF2/Argon2. Never persist the derived key — require re-derivation on each session. This protects data at rest against exfiltration by compromised code that runs before the user authenticates.
- **External verification via browser extension**: For high-security deployments, a WEBCAT-style or Code Verify-style extension provides an independent trust anchor outside the compromised origin's control.

This combination provides **hours of protection** against a compromised server for existing users (until the SW update check succeeds), **detection capability** through hash and signature verification, **data-at-rest protection** through encryption, and **independent verification** through the external extension.

---

### Conclusion: where the web model fundamentally breaks

The web's security architecture was built on an implicit contract: **the origin server is the application**. The browser trusts it unconditionally. SRI protects against compromised third parties. CSP protects against injection. HTTPS protects the transport. But none of these mechanisms was designed for the threat of a compromised first party, because in the web model, compromising the server *is* compromising the application — there's nothing left to protect.

Native applications break this coupling. Code is signed by the developer, verified by the OS, installed once, and persists independently of the distribution server. DigiCert A compromised server can serve malicious *updates*, but these must pass app store review and OS-level signature verification. The server cannot silently replace already-installed code. This is precisely why Signal refuses to offer a web client. Aboutsignal

Three changes to the browser platform could meaningfully close this gap. First, **SRI for service worker registration** (the proposed `integrity` field in `register()`) would create a browser-enforced root of trust, GitHub preventing SW replacement unless the new script matches a pinned hash. GitHub Second, **browser-native code transparency** — the Proton/Huigens and WEBCAT proposals brought into the browser engine — would eliminate the need for extensions. W3C Third, **signed web bundles with cross-browser support** (the IWA model) would separate code signing from code hosting entirely. GitHub Chrome Developers

Until these primitives ship, the honest assessment is: **a well-defended PWA buys hours, not guarantees**. The service worker is a speed bump, not a wall. HackTricks For applications where server compromise is in the threat model — whistleblowing platforms, E2E encrypted messaging, cryptocurrency wallets — the responsible choice remains native application distribution with OS-level code signing. The web's openness and reach come at the cost of its trust model. Tonyarcieri That tradeoff is architectural, not incidental, and no amount of defense-in-depth can fully overcome it within the current browser security model.